

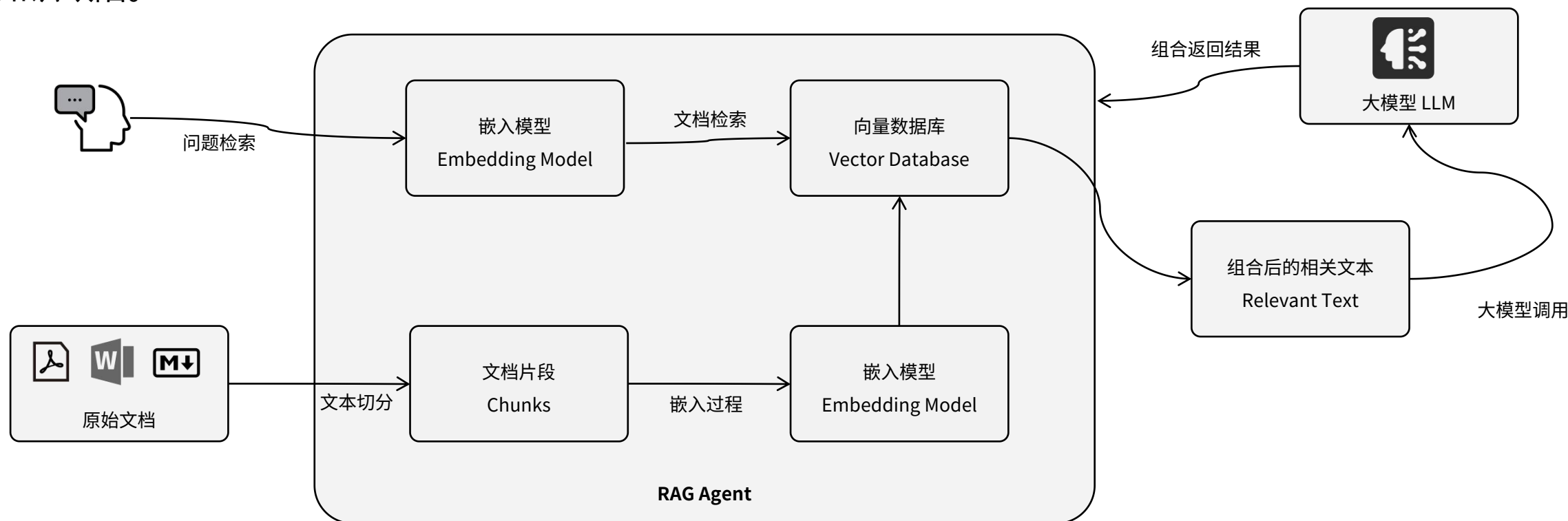
RAG 的工程实现 在线研讨会分享

扫描加群（请备注来意）



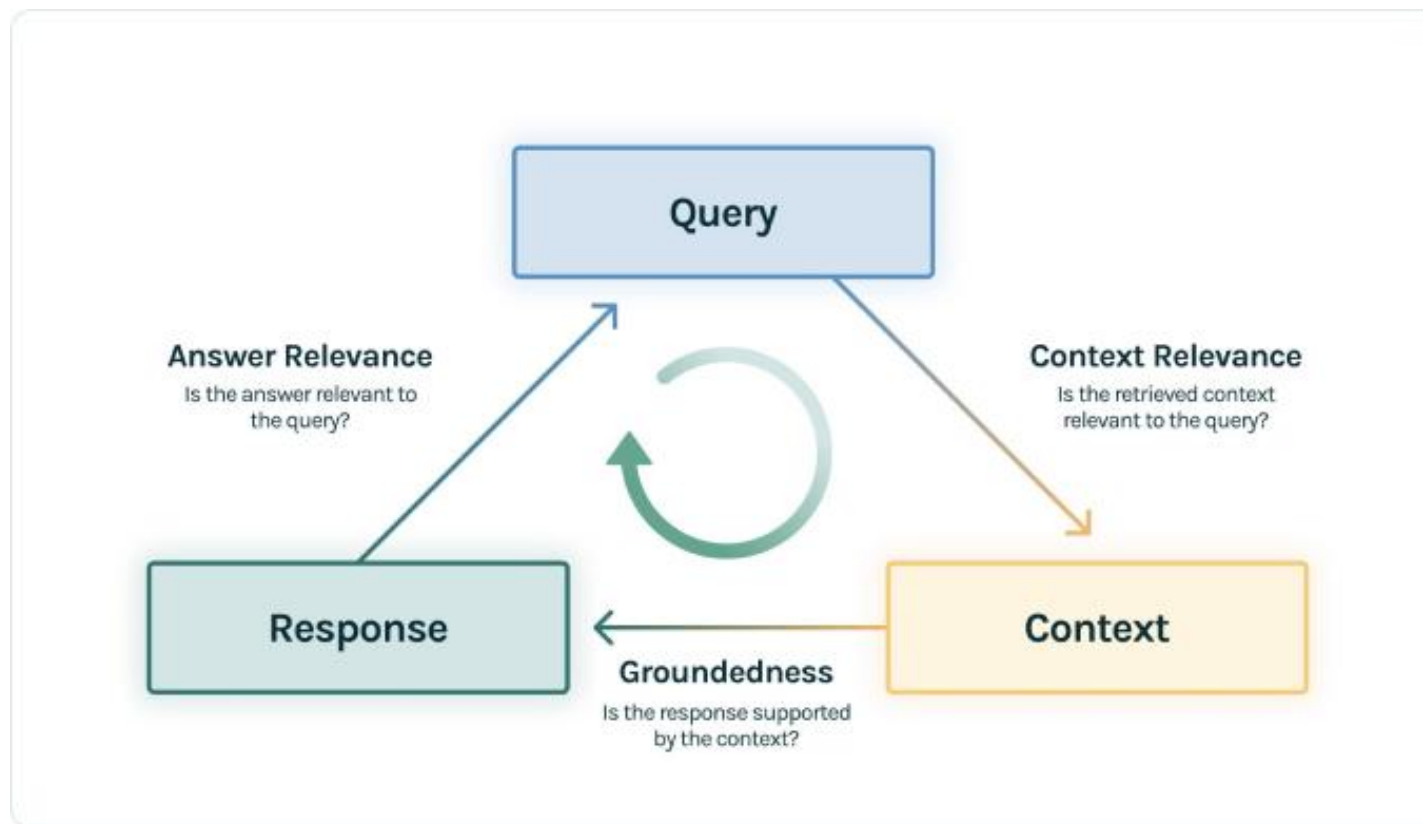
什么是 RAG?

RAG (Retrieval-Augmented Generation) 解决的问题是：让大模型在回答问题时能够实时检索外部知识，从而减少幻觉并利用最新或私有数据。RAG 本质是在“不给模型重新训练”的情况下，弥补预训练模型的知识缺陷。



RAG 哲学？

RAG 其实是在组合上下文相关性和答案相关性。



RAG 的基本概念

RAG 本身：

Retrieval-Augmented Generation 指的是一种架构：在大模型生成答案之前，先从外部知识库中检索相关内容，再把这些内容作为上下文提供给模型，从而提升回答的准确性。

Text Embedding 文本嵌入：

Text Embedding 指把一段文本映射成一个高维向量，使语义相近的文本在向量空间中距离更近。这样就可以通过数学方式判断“内容是否相似”。

Text Embedding 向量数据库：

Vector Database 用来存储文本向量，并支持基于相似度的搜索，例如寻找“最接近用户问题的几段文档”。

Semantic Search 相似度搜索：

Semantic Search 指根据语义相似度而不是关键词匹配来查找内容，这是 RAG 与传统搜索系统的重要区别。

Top-K Retrieval：

Top-K Retrieval 表示系统会选出最相关的 K 个片段，并把它们一起提供给大模型作为上下文。

Prompt 提示词：

Prompt Engineering 指如何把用户问题和检索到的文档组合成一个输入，让大模型能够正确理解上下文并生成回答。

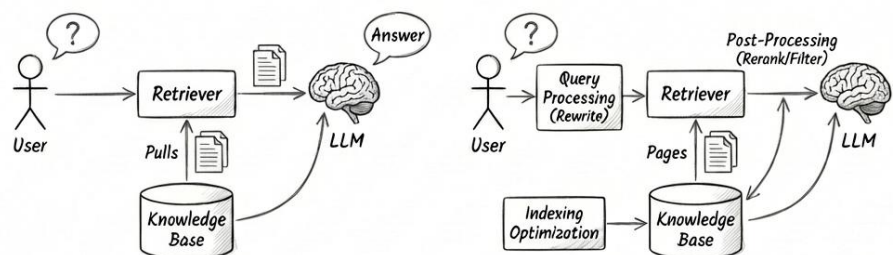
Context Window：

Context Window 表示模型一次能够读取的最大文本长度。RAG 系统需要控制检索内容的数量，否则可能超出模型的输入限制。

Rerank：

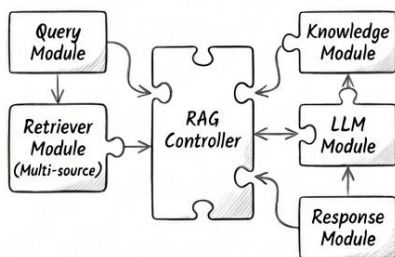
Reranking Model 指在初步检索后，再使用一个模型对结果重新排序，从而把最相关的文档排在最前面。

RAG 的几个变体

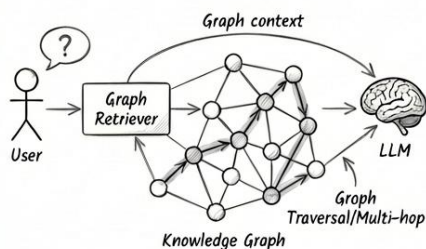


Naive RAG: Basic pipeline

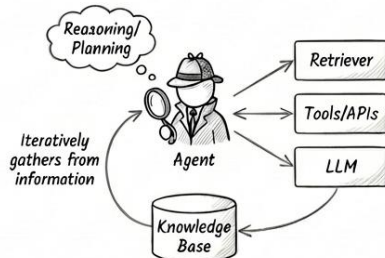
Advanced RAG: Pre/During/Post Optimization



Modular RAG: Pluggable & Scalable



Graph RAG: Multi-hop & Rich Context



Agentic RAG: Intelligent Analysis & Autonomy

Naive RAG: 最基础的 RAG 实现。

Advanced RAG: 在 Naive RAG 基础上，对检索前、检索、检索后分别进行优化。

Modular RAG: 更工程化 RAG 实现，模块化可扩展。

Graph RAG: 利用图检索能力，增强 multi-hop 检索和丰富上下文。

Agentic RAG: 利用 Agent 能力，使 RAG 具备智能分析与思考能力，大幅增强检索效果。

业界主流目前还是 **Advanced RAG** 为主，目标在为特定的场景、知识库进行优化。

为什么要优化的例子

优化方法：Query Rewrite

用户输入：龙虾怎么配置？

用户真实想法：如何配置和搭建 OpenClaw（俗称小龙虾）？

上下文分析

结合用户最近的操作/浏览历史，把“龙虾”识别为技术概念“OpenClaw 小龙虾”而非食物。

同义词扩展

添加相关技术关键词，例如“安装”“部署”“配置”“服务”。

问法规范化

缩写还原或自然语言规范化，例如把“怎么配置”改为“部署和配置指南”。

重写后的查询：OpenClaw 小龙虾部署与配置指南

RAG 的核心技术组件选型

技术组件	作用	常见选型	适用场景
文档解析	将 PDF、HTML、Word 等内容解析成文本	Apache Tika、Unstructured、LlamaParse	企业知识库、文档系统
文档切分 (Chunking)	将长文档切分为可检索的文本片段	LangChain TextSplitter、LlamaIndex Node Parser	所有 RAG 系统
Embedding 模型	将文本转换为语义向量	OpenAI Embeddings、Sentence-BERT、BGE	语义搜索、知识库问答
向量数据库	存储向量并进行相似度搜索	Pinecone、Milvus、Weaviate、Chroma	大规模知识库
检索引擎	根据用户问题找到相关文档	向量检索、BM25、Hybrid Search	复杂知识检索
Rerank 模型	对检索结果重新排序，提高相关性	Cohere Rerank、BGE Reranker	高精度问答
RAG 框架	负责检索流程与上下文拼接	LangChain、LlamaIndex、Haystack	快速构建 RAG 应用
LLM 模型	负责生成最终答案	GPT-4、Claude、Llama 3	问答、总结、推理
Agent / Workflow	复杂任务编排与多步骤检索	LangGraph、AutoGen	多步骤知识查询
API 封装 / Web 框架	将 RAG 能力封装为 HTTP API	FastAPI、Flask、Spring Boot、Express.js	提供对外服务接口

RAG 工程的实现流程



数据解析和拆分

处理结构化或非结构化数据，载入到向量数据库。



相似性检索

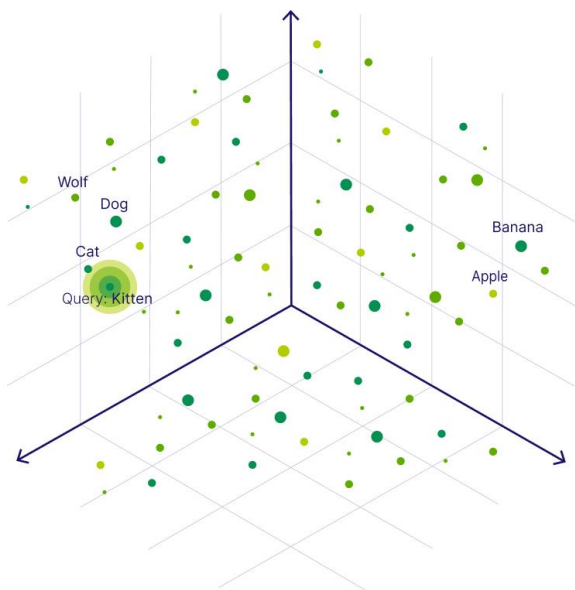
根据用户的输入，检索相关的数据条目，并进行重排序的优化。



提示词工程

使用合适的提示词，结合召回的数据，发送到 LLM 获得最终结果。

相似性检索



向量搜索的本质是匹配两个句子之间的**语义相关性**。

原始知识库句子

1. 苹果手机电池寿命如何延长?
2. iPhone 如何省电?
3. Android 手机充电慢怎么办?
4. 如何提高笔记本电脑性能?

用户搜索

1. 怎么让 iPhone 更省电?

向量解码

2. iPhone 如何省电?

向量化

向量数据库

- 句子1 $\rightarrow [0.12, -0.41, 0.77, \dots]$
句子2 $\rightarrow [0.15, -0.39, 0.75, \dots]$
句子3 $\rightarrow [-0.30, 0.22, 0.51, \dots]$
句子4 $\rightarrow [0.44, -0.10, -0.32, \dots]$

用户查询向量

query $\rightarrow [0.14, -0.40, 0.74, \dots]$

匹配到的向量结果（余弦相似度）

句子2 $\rightarrow [0.15, -0.39, 0.75, \dots]$

数据拆分和Embedding



Embedding 模型对比

模型	发布机构	向量维度	上下文长度	模型大小	中文能力	多语言	本地部署	典型场景
Sentence-BERT	UKP Lab	384 / 768	512	小	一般	一般	容易	语义相似度、简单搜索
BGE	北京智源	384 / 768 / 1024	512	中	很强	支持	容易	RAG、向量搜索
BGE-M3	北京智源	1024	8192	大	很强	很强	中	多语言检索
E5	Microsoft	768 / 1024	512	中	较好	很强	容易	多语言语义搜索
GTE	阿里巴巴	768	512	中	较好	支持	容易	企业搜索系统
Jina Embeddings	Jina AI	768 / 1024	8192	中	较好	很强	中	长文本检索
text-embedding-3-large	OpenAI	3072	8192	云模型	很好	很强	API	高质量语义搜索

Embedding 模型演进

词向量阶段 (Word Embedding)

代表模型：Word2Vec、GloVe、FastText

训练方式：自监督 (Self-Supervised)

特点：

- 单词级别 embedding
- 捕捉语义相似性 (king - man + woman $\approx$ queen)
- 不能直接处理句子或文档

句子/段落向量阶段 (Sentence / Paragraph Embedding)

代表模型：Paragraph Vector / Doc2Vec / InferSent / Universal Sentence Encoder (USE)

训练方式：自监督 + 句级预测

特点：

- 句子或段落级别 embedding
- 捕捉句子语义，而非单词
- 适合语义相似度计算

对比学习阶段 (Contrastive Learning / Bi-Encoder)

代表模型：SBERT、SimCSE、E5、BGE

训练方式：对比学习 (Contrastive Learning)

特点：

- 双塔结构 (Query / Document 双独立编码)
- 高效支持大规模语义搜索
- 可以无监督 (SimCSE unsup) 或有监督 (SimCSE sup)

大模型融合阶段 (LLM-based Embedding)

代表模型：OpenAI text-embedding-3 系列、Claude Embedding、BGE

训练方式：自监督 + 对比学习 + 大规模预训练

特点：

- 支持更复杂语义表达
- 适合 QD 搜索场景
- 可微调或直接用于 RAG

文档解析工具对比

工具	类型	支持格式	表格解析	OCR	是否开源	特点	适用场景
PyMuPDF / pdfplumber	基础解析库	PDF	弱	否	是	速度快，直接提取文本	简单 PDF
Apache Tika	通用解析	1000+ 文件	中	否	是	Java 生态，支持大量格式	企业文档系统
Unstructured	结构化解析	PDF / DOCX / HTML / Email / Image	中-强	是	是	能识别 title / list / table 等元素	通用 RAG ingestion
Docling	AI 结构解析	PDF / Office / Image	强	是	是	layout + table AI 模型	高质量知识库
LlamaParse	AI 解析服务	PDF（复杂表格）	强	部分	否	输出 markdown, 适合 RAG	财报 / 报告
LangChain Loader	Loader 框架	多种	依赖外部	依赖外部	是	ingestion 框架, 不是解析器	pipeline 组件
BeautifulSoup	HTML 解析	HTML	不涉及	否	是	web 数据提取	爬虫知识库

文档解析难点

内容不完整或者错误

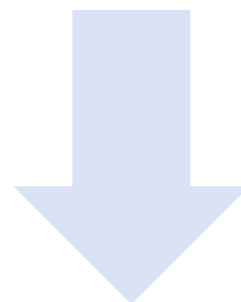
- 文档整体被切断
- 文档部分内容丢失
- 表格、文本、图片混排
- 段落结构不标准

文档格式处理

- 最容易处理：Markdown、Text、HTML
- 相对容易处理：PDF、Word、PPT
- 非文档内容：Image、Video、Audio 需要做转写的内容

特殊场景

- PDF 单元格合并
- 代码块识别
- 跨栏 PDF 处理
- 表格跨页



解决策略

结构化文档：Unstructured

图文混排文档：使用 AI 解析服务（例如腾讯向量数据 AI 套件）

音频、视频：转写 + AI 解析

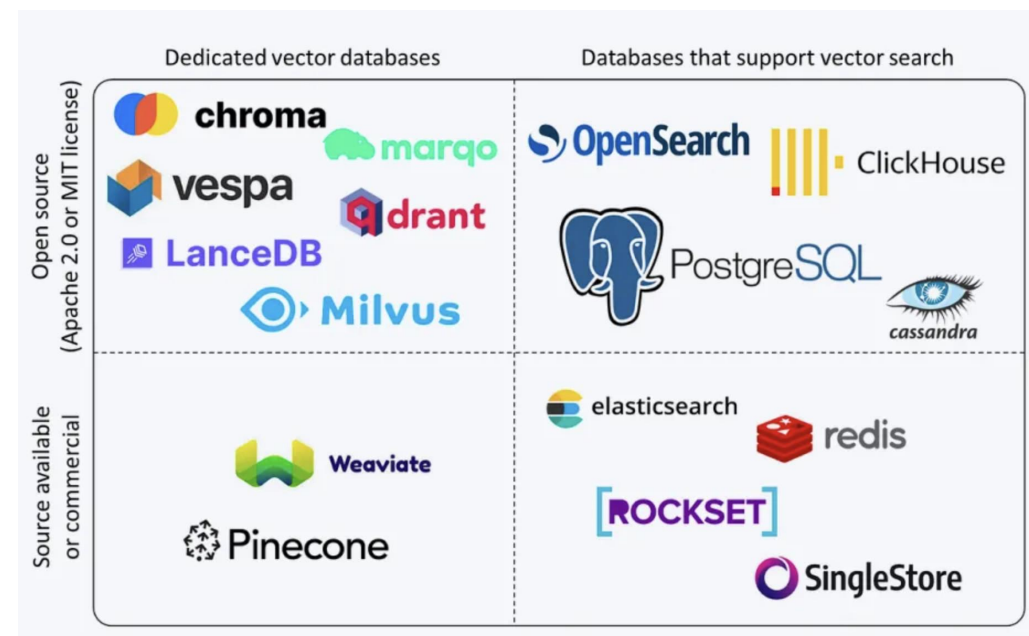
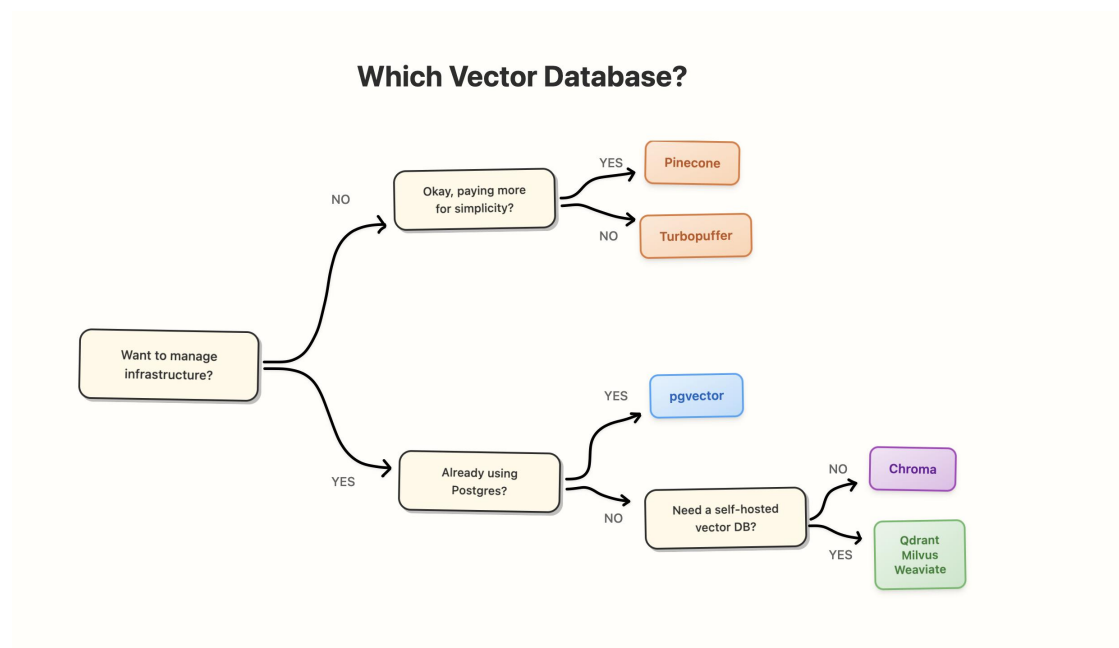
文档拆分策略对比

拆分方式	LangChain 支持	主要实现类	拆分逻辑	优点	缺点	适用场景
固定长度拆分	是	CharacterTextSplitter	按固定字符数切分	实现最简单、速度快	容易破坏语义	简单文本、测试
Token 拆分	是	TokenTextSplitter	按 token 数量切分	与 LLM 上下文限制一致	仍可能切断语义	LLM 长上下文控制
递归拆分	是	RecursiveCharacterTextSplitter	按段落→行→空格递归拆分	尽量保持语义完整	控制逻辑略复杂	大多数 RAG 系统
句子拆分	是	NLTKTextSplitter	按句子边界拆分	语义自然	长句会超长	文章、书籍
Markdown 结构拆分	是	MarkdownHeaderTextSplitter	按 Markdown 标题层级拆分	保留文档结构	需要结构化文档	技术文档、博客
HTML 结构拆分	是	HTMLHeaderTextSplitter	按 HTML header 或 DOM 结构拆分	结构清晰	HTML 解析复杂	爬虫内容、网页
代码结构拆分	是	PythonCodeTextSplitter	按函数、类拆分	保持代码逻辑完整	只适用于代码	代码知识库
语义拆分	部分支持	SemanticChunker	通过 embedding 判断语义变化	chunk 语义集中	计算成本高	高质量知识库
滑动窗口拆分	是（参数实现）	chunk_overlap 参数	chunk 之间保留重叠部分	防止信息断裂	增加数据量	通用优化
多粒度拆分	需要自定义	自定义 pipeline	同时生成大小不同 chunk	提高召回率	复杂度高	大型知识库

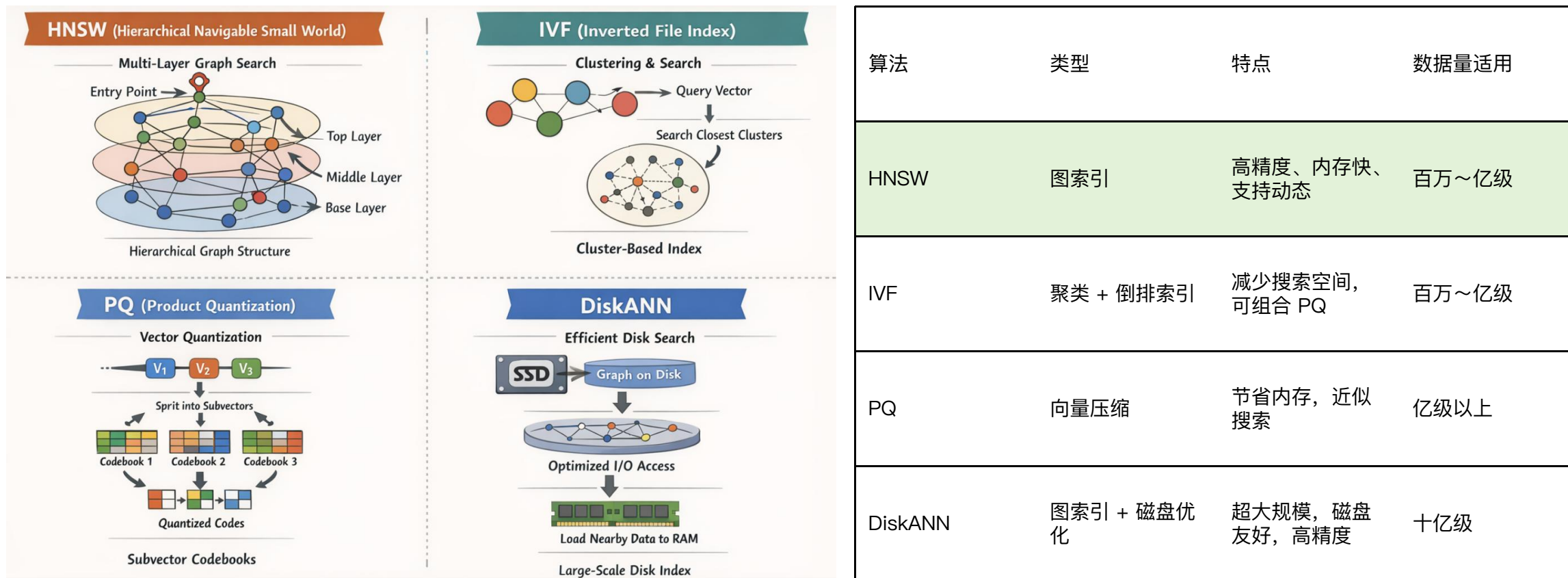
向量数据库对比

维度	Pinecone	Milvus	Weaviate	Chroma	Milvus Lite
产品类型	托管向量数据库（SaaS）	分布式向量数据库	向量搜索引擎	嵌入式向量数据库	嵌入式向量数据库
开源情况	否	是	是	是	是
部署方式	云托管	Docker / Kubernetes / Cloud	Docker / Kubernetes	本地库	本地库
架构模式	Serverless / 托管服务	分布式存储 + 查询节点	分布式搜索架构	单机嵌入式	单机嵌入式
运维复杂度	无需运维	高	中	极低	极低
扩展能力	自动扩容	水平扩展	水平扩展	基本没有	基本没有
数据规模能力	百万到十亿级	十亿级以上	百万到亿级	万到百万级	万到百万级
索引算法	HNSW / 专有优化	HNSW / IVF / PQ / DiskANN	HNSW	HNSW	HNSW
向量类型支持	Dense	Dense / Sparse	Dense / Hybrid	Dense	Dense
Hybrid Search	支持	支持	强支持	较弱	较弱
Metadata 过滤	强	强	非常强	中	中
查询延迟	低	低	中	中	中
写入吞吐	中	高	中	低	低
GPU 加速	否	支持	否	否	否
SDK 语言	Python / JS / Go	Python / Go / Java / Node	Python / JS / Go	Python	Python
LangChain 支持	支持	支持	支持	支持	支持
LlamaIndex 支持	支持	支持	支持	支持	支持
多租户能力	强	中	强	弱	弱
权限控制	强	中	中	无	无
数据持久化	云存储	本地 / 对象存储	本地 / 云	SQLite / DuckDB	本地文件
典型使用场景	AI SaaS 产品	大规模向量检索系统	搜索与知识库系统	RAG 原型开发	本地开发测试

向量数据库选择



向量数据库索引原理



RAG 中上下文整合策略

在不同的场景下选择合适的提示词策略，融合输出。

直接拼接 (Concatenate)

检索到的 top-K 文档直接拼在查询后面，作为输入给 LLM。

特点：简单直接，容易实现，但是可能超过模型上下文长度，文档冗余影响生成。

```
from langchain.chains import RetrievalQA
from langchain.prompts import PromptTemplate

template = """
根据以下文档回答问题:
{context}

问题:
{question}
"""
prompt = PromptTemplate(template=template, input_variables=["context", "question"])

qa = RetrievalQA.from_chain_type(
    llm=llm,
    retriever=retriever,
    chain_type="stuff", # 把 top-K 文档直接拼接
    chain_type_kwargs={"prompt": prompt}
)
```

摘要融合 (Refine)

在已有答案基础上，加入新的文档内容优化答案。

特点：可以减少输入长度，适合上下文长度有限或文档较长的场景。

```
qa = RetrievalQA.from_chain_type(
    llm=llm,
    retriever=retriever,
    chain_type="refine",
    chain_type_kwargs={
        "question_prompt": question_prompt,
        "refine_prompt": refine_prompt
    }
)
```

分块生成 (Chunked)

将检索文档分块，逐块生成融合答案。

特点：适合超大文档库。

```
qa = RetrievalQA.from_chain_type(
    llm=llm,
    retriever=retriever,
    chain_type="map_reduce",
    chain_type_kwargs={
        "map_prompt": map_prompt,
        "combine_prompt": combine_prompt
    }
)
```

RAG 中的记忆策略—短期记忆

拼接历史 (Buffer Memory)

把历史对话直接加入 prompt。

特点：简单直接，容易超 context，噪音多。

```
from langchain.memory import ConversationBufferMemory
from langchain.chains import ConversationChain
from langchain.chat_models import ChatOpenAI

llm = ChatOpenAI()

memory = ConversationBufferMemory()

conversation = ConversationChain(
    llm=llm,
    memory=memory,
    verbose=True
)

conversation.run("我叫 Lisa")
conversation.run("我刚刚说我叫什么? ")
```

滑动窗口 (Window Memory)

保留最近 N 轮对话。

特点：控制上下文长度，根据时间遗忘。

```
from langchain.memory import ConversationBufferWindowMemory

memory = ConversationBufferWindowMemory(k=2)

conversation = ConversationChain(
    llm=llm,
    memory=memory,
    verbose=True
)

conversation.run("我叫 Lisa")
conversation.run("我住在上海")
conversation.run("我喜欢编程")
conversation.run("我刚刚说我住哪? ")
```

摘要记忆 (Summary Memory)

用 LLM 把历史压缩成摘要。

特点：节省 token，有信息损失。

```
from langchain.memory import ConversationSummaryMemory

memory = ConversationSummaryMemory(llm=llm)

conversation = ConversationChain(
    llm=llm,
    memory=memory,
    verbose=True
)

conversation.run("我叫 Lisa, 我是一个程序员")
conversation.run("我平时写 Node.js 和 AI 应用")
conversation.run("总结一下我是谁? ")
```

混合记忆 (Summary + Buffer)

新对话先进 Buffer，超过阈值后，把旧对话压缩成 Summary。

特点：结合前面的特点。

```
from langchain.memory import ConversationSummaryBufferMemory

memory = ConversationSummaryBufferMemory(
    llm=llm,
    max_token_limit=1000
)

conversation = ConversationChain(
    llm=llm,
    memory=memory,
    verbose=True
)

conversation.run("我叫 Lisa")
conversation.run("我在做一个 RAG 系统")
conversation.run("我在研究向量数据库")
conversation.run("总结一下我在做什么? ")
```

RAG 中的记忆策略—长期记忆

向量记忆 (Vector Store Memory)

把历史对话 / 用户信息 embedding, 存入向量数据库, 每次 query 时检索相关记忆。

特点: 可扩展 (无限记忆), 语义检索 (不是时间顺序)。

对话查询重写 (Query Rewriting)

把“当前问题 + 历史对话”改写成完整问题。

特点: 控制上下文长度, 根据时间遗忘, 可以大大提高检索的精确度。

记忆检索 + 文档检索融合

同时检索两类信息, 知识库 (RAG 文档), 用户记忆。

特点: $\text{context} = \text{docs} + \text{memory}$, 可以增加权重。

Key-Value Memory (结构化记忆)

不用embedding 把记忆清洗成用户的画像/偏好信息。

特点: 需要额外的存储和 Token 消耗。

知识图谱记忆 (KG Memory)

用图结构存储记忆。

特点: 适合复杂关系推理。

怎么评估 RAG 的质量？

RAG 结果质量 = Parsing（文档解析） + Embedding（向量化） + Chunk（切分） + Rerank（精排）

评估维度	子指标	描述	常用方法 / 指标
检索效果	Recall@K	前 K 条检索结果中是否包含正确答案	Recall@K
	Precision@K	前 K 条结果中相关内容占比	Precision@K
	MRR（Mean Reciprocal Rank）	正确答案出现在排名第几位的平均倒数	MRR
	NDCG（Normalized DCG）	考虑相关性和排名的综合评分	NDCG@K
生成效果	BLEU / ROUGE / METEOR	与参考答案文本的词或句子重叠度	自动计算
	BERTScore / MoverScore	基于语义的相似度	自动计算
	Factual Accuracy	答案是否符合检索到的事实	人工或自动对比事实库
	Answer Exact Match	是否完全匹配参考答案（QA任务）	自动计算
	Fluency	文本语言是否自然可读	人工评估
	Relevance	答案是否切中查询意图	人工评估
	Coverage	答案是否充分利用检索信息	人工评估
系统指标	Latency / QPS	查询响应时间和系统吞吐量	测试负载下测量
	Robustness	对长查询、模糊查询的表现	实验测试
	Hallucination Rate	虚假或不可靠信息比例	人工标注或事实对比

QQ 和 QD 问题

QQ 问题 (Query-Query)

用一个“问题”，去匹配“另一个问题”，本质是语义对齐。

例如：

iPhone 怎么省电？匹配问题库：

- 苹果手机如何省电？
- iPhone 电池优化方法
- 如何延长苹果手机续航？

VS

QD 问题 (Query-Document)：

用“问题”，去匹配“答案型文档”，本质是语义推理。

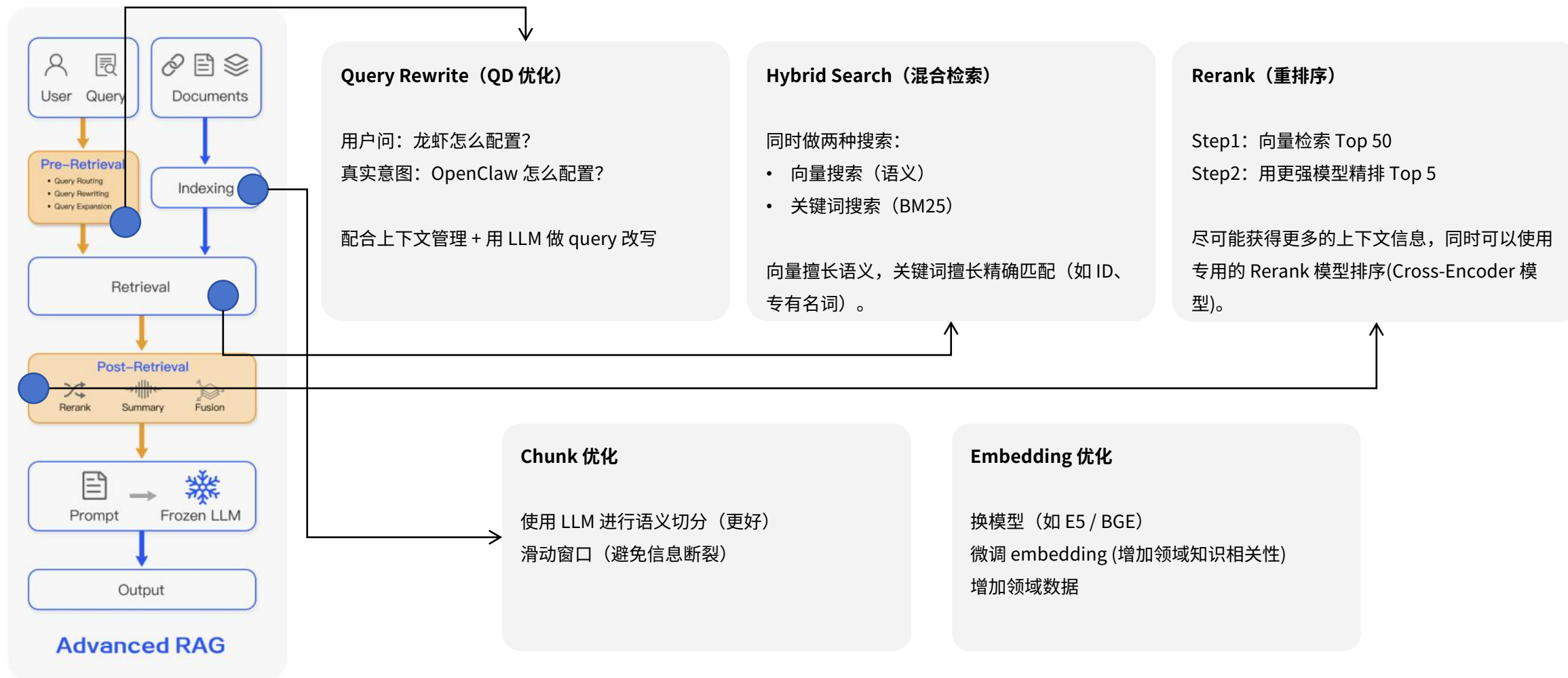
例如：

iPhone 怎么省电？匹配问题库：

- 关闭后台应用刷新可以显著降低功耗
- iPhone 电池优化方法
- 苹果手机增加巡航的方法

QD 搜索的本质，是把“问题”和“知识”投影到同一个语义空间，再通过距离来完成匹配。

RAG 中的优化策略



为什么要使用专门的模型 Rerank?

因为双塔模型更适合使用向量数据库快速大规模查询，而 Cross-Encoder 能实现小规模数据的逻辑理解，比如判断一对句子的关系是“蕴含”（A推出B）、“矛盾”还是“中立”。这需要非常精细的逻辑理解，Cross-Encoder 是首选。

维度	双塔结构	Cross-Encoder
处理方式	独立编码，后交互。	联合编码，深度融合。
输入	句子A -> 塔1 -> 向量u； 句子B -> 塔2 -> 向量v。	[CLS] 句子A [SEP] 句子B [SEP]
信息交互	晚交互。只在最后计算向量距离时交互，前期没有任何信息往来。	早交互。从第一层开始，两个句子的信息就在互相“关注”和“融合”。
计算效率	极高。可以预计算所有句子的向量，存入数据库。查询时只需计算一次查询向量，然后做向量检索。适合大规模场景。	较低。每一对句子都必须完整计算一次，无法预计算或缓存。如果有n个句子要找最相似的，需要计算n次。
准确率	相对较高。虽然效果好，但因为交互有限，可能会丢失一些细微的语义关联。	非常高。通常被认为是这类任务的准确率上限，因为它捕捉到了最全面的交互信息。
典型应用	大规模语义搜索、文本聚类、推荐系统召回阶段（从千万级数据中快速筛选出几百个候选）。	精排、小规模数据对的分析、自然语言推理（如判断两句话是否矛盾）。

Rerank 策略

Pointwise Re-rank

用 LLM 或小模型对每个文档打分。

特点：简单直接，容易实现，速度快，但是缺点是不考虑文档之间的关系，适合粗排。

例如：

Query: "向量数据库索引原理"

Doc1 → 0.92

Doc2 → 0.75

Doc3 → 0.30

Pairwise Re-rank

比较两个文档哪个更相关，最后通过投票 / 排序算法得到全局排序。

特点：有文档之间的相关性信息，介于 Pointwise 和 Listwise 之间，成本比较高。

例如：

DocA vs DocB → A 更相关

DocA vs DocC → C 更相关

然后最终排序。

Listwise Re-rank

一次性让模型看全部候选。

特点：全局最优，最符合真实排序目标（NDCG, Normalized Discounted Cumulative Gain），token 成本高，长度受限，适合精排，或者有专用的排序模型。

例如（提示词）：

以下是5个文档，请按相关性排序：

Doc1

Doc2

.....

主流实践：召回（TopK）→ 粗排（Pointwise）→ 精排（Listwise / 小范围），判断标准：

候选数量 ≤ 5 → 可以 Listwise

候选数量 5~20 → 混合（Point + List）

Rerank 模型选择

维度	Cohere Rerank	BGE Reranker	Jina Reranker	ColBERT v2
模型类型	商业 API（Cross-encoder）	开源（Cross-encoder）	商业 API / 开源（Hybrid）	开源（Late Interaction）
部署方式	SaaS / 私有化	本地（GPU/CPU）	SaaS + 可本地	本地部署
精度（NDCG）	高（行业标杆）	中高（接近可用）	中高（接近 Cohere）	高（长文本优势明显）
多语言能力	很强	中等	强	中等
延迟	中（API）	低（本地批处理）	中	较高
吞吐能力	中	高	中	中低
成本	高（按调用）	低（算力成本）	中	中（算力成本）
可控性	低（黑盒）	高（可微调）	中	高
长文本能力	中	中	中	很强
工程复杂度	低	中	中	高
典型优势	稳定、高精度	成本低、可控	平衡型	细粒度匹配强